

A Tutorial on Proximal Policy Optimization

From Policy Gradients to PufferLib

Contents

1	Nomenclature and Problem Setup	3
1.1	Trajectories	3
1.2	Policy	3
1.3	Total Discounted Return	3
1.4	Value and Advantage Functions	3
1.5	Utility Function	4
2	The Policy Gradient Theorem	4
2.1	Deriving the Gradient of the Utility	4
2.2	Monte-Carlo Estimation	5
3	Variance Reduction	5
3.1	Reward-to-Go	5
3.2	Baseline Subtraction	5
3.3	Actor-Critic Methods	5
3.4	Generalised Advantage Estimation (GAE)	6
4	Trust Region Policy Optimisation (TRPO)	6
4.1	Motivation	6
4.2	The Surrogate Objective	6
4.3	The KL Constraint	7
5	Proximal Policy Optimisation (PPO)	7
5.1	The Clipped Surrogate Objective	7
5.2	Value Function Loss	7
5.3	Entropy Bonus	8
5.4	Full PPO Objective	8
5.5	Implementation	8
6	Extensions in PufferLib	8
6.1	V-Trace Importance Weighting	8
6.2	Priority Experience Replay	10
6.3	Clipped Surrogate and Value Losses	10
6.4	Gradient Clipping and Learning Rate Schedule	11
7	Training Diagnostics: Understanding the Log	12

1 Nomenclature and Problem Setup

We consider the standard reinforcement learning (RL) setting: an agent interacts with an environment over discrete time steps, observing states and selecting actions, with the goal of maximizing accumulated reward.

1.1 Trajectories

A *trajectory* (or *rollout*) of horizon H is the sequence

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_H, a_H, r_H), \quad (1)$$

where $s_t \in \mathcal{S}$ is the state, $a_t \in \mathcal{A}$ the action, and $r_t \in \mathbb{R}$ the scalar reward received at step t .

1.2 Policy

A *stochastic policy* π_θ is a conditional distribution over actions given states, parameterised by a vector θ :

$$\pi_\theta(a \mid s) = \Pr_\theta[A_t = a \mid S_t = s]. \quad (2)$$

In deep RL this is typically a neural network with weights θ .

1.3 Total Discounted Return

Given a discount factor $\gamma \in [0, 1)$, the *discounted return* of a trajectory is

$$R(\tau) = \sum_{t=0}^H \gamma^t r_t. \quad (3)$$

The discount factor down-weights future rewards, both for mathematical convenience (ensuring convergence) and to reflect temporal preference.

1.4 Value and Advantage Functions

Definition 1.1 (State-value function). $V^\pi(s) = \mathbb{E}_\pi[R(\tau) \mid s_0 = s]$, the expected return when starting in state s and following π thereafter.

Definition 1.2 (Action-value function). $Q^\pi(s, a) = \mathbb{E}_\pi[R(\tau) \mid s_0 = s, a_0 = a]$.

Definition 1.3 (Advantage function).

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \quad (4)$$

The advantage measures how much better action a is compared to the average action taken by π in state s .

1.5 Utility Function

The quantity we want to maximise is the *utility*

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^H \gamma^t r_t \right]. \quad (5)$$

Since the expectation is taken over trajectories sampled from the current policy, both the sampling distribution *and* the rewards depend on θ , making direct optimisation non-trivial.

2 The Policy Gradient Theorem

2.1 Deriving the Gradient of the Utility

To apply first-order (gradient) methods we need $\nabla_\theta J(\theta)$. The key observation is the *log-derivative trick* (also called the REINFORCE trick):

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s). \quad (6)$$

The probability of a trajectory under policy π_θ factorises as

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^H \pi_\theta(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t), \quad (7)$$

so the log-probability decomposes as

$$\log p_\theta(\tau) = \log p(s_0) + \sum_{t=0}^H \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^H \log p(s_{t+1} \mid s_t, a_t). \quad (8)$$

Only the middle sum depends on θ , hence

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^H \nabla_\theta \log \pi_\theta(a_t \mid s_t). \quad (9)$$

Applying the log-derivative trick to the utility gives

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau \\ &= \int \nabla_\theta p_\theta(\tau) R(\tau) d\tau \\ &= \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) R(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log p_\theta(\tau)]. \end{aligned} \quad (10)$$

Substituting the factorisation of $\nabla_\theta \log p_\theta(\tau)$ yields the **REINFORCE policy gradient estimator**:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^H \nabla_\theta \log \pi_\theta(a_t \mid s_t) R(\tau) \right]}. \quad (11)$$

2.2 Monte-Carlo Estimation

In practice we approximate Eq. (11) with a sample average over N trajectories $\{\tau^{(i)}\}_{i=1}^N$:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) R(\tau^{(i)}). \quad (12)$$

This estimator is unbiased but can have very high variance, which motivates the modifications described in Section 3.

3 Variance Reduction

High variance in the policy gradient estimate leads to unstable training. Three standard techniques are used to reduce it.

3.1 Reward-to-Go

Using the full return $R(\tau)$ introduces unnecessary noise: actions at step t cannot influence rewards received *before* t . Replacing $R(\tau)$ with the *reward-to-go*

$$\hat{R}_t = \sum_{t'=t}^H \gamma^{t'-t} r_{t'} \quad (13)$$

gives an estimator with the same expectation but lower variance:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{R}_t \right]. \quad (14)$$

3.2 Baseline Subtraction

Adding any function $b(s_t)$ that does not depend on the action does not bias the gradient (since $\mathbb{E}_a[\nabla_{\theta} \log \pi_{\theta}(a|s) b(s)] = 0$), but can reduce variance. A natural choice is the state-value function $V^{\pi}(s_t)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (\hat{R}_t - V^{\pi}(s_t)) \right]. \quad (15)$$

The term $\hat{R}_t - V^{\pi}(s_t)$ approximates the advantage $A^{\pi}(s_t, a_t)$. In practice we use a learned critic $V_{\phi}(s)$ to estimate the baseline.

3.3 Actor-Critic Methods

Baseline subtraction (Section 3) still uses a Monte-Carlo estimate of the return: the gradient update is computed only *after* a full trajectory has been collected, and the baseline $V_{\phi}(s_t)$ reduces variance without contributing to the return estimate itself.

An *actor-critic* method goes one step further by letting the critic *replace* the Monte-Carlo return with a bootstrapped estimate. The policy (the *actor*) is updated using a one-step TD advantage:

$$\hat{A}_t^{\text{TD}} = r_{t+1} + \gamma V_{\phi}(s_{t+1})(1 - d_{t+1}) - V_{\phi}(s_t), \quad (16)$$

where V_ϕ is the learned critic. Both components are trained concurrently: the actor maximises expected return via the policy gradient, while the critic minimises a regression loss against TD targets.

Key difference from REINFORCE with a baseline. In REINFORCE with a baseline the value function V_ϕ appears *only* as a control variate—it does not enter the return estimate. The policy gradient still uses the full sum of future rewards $\hat{R}_t$, which is unbiased but high-variance. An actor-critic method substitutes $\hat{R}_t$ with the (lower-variance but biased) bootstrapped target in Eq. (16). The trade-off is:

	REINFORCE + baseline	Actor-critic (TD)
Bias	None (unbiased gradient)	Introduced by bootstrapping
Variance	High (full MC return)	Lower (single-step TD)
Updates	After full episode	After each step (online)

Because the critic bootstraps from its own (imperfect) predictions, approximation bias accumulates when the value function is inaccurate. The choice of how many steps to unroll before bootstrapping controls the bias-variance trade-off, which Generalised Advantage Estimation formalises as a continuous interpolation via the λ parameter.

3.4 Generalised Advantage Estimation (GAE)

Schulman et al. [2] introduced GAE as a low-variance, low-bias estimate of the advantage. For a λ -weighted mixture of k -step TD errors $\delta_t^{(k)} = r_t + \gamma V(s_{t+1}) - V(s_t)$, the GAE estimator is

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad (17)$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1})(1 - d_{t+1}) - V(s_t)$ is the one-step TD residual and $d_{t+1} \in \{0, 1\}$ indicates episode termination.

Setting $\lambda = 0$ recovers the TD(0) advantage (high bias, low variance); $\lambda = 1$ recovers the Monte-Carlo advantage (low bias, high variance). Intermediate values trade off between the two.

4 Trust Region Policy Optimisation (TRPO)

4.1 Motivation

A fundamental problem with vanilla policy gradient is that a step too large in parameter space can collapse performance catastrophically and irreversibly, because the new sampling distribution is far from the one used to collect data.

4.2 The Surrogate Objective

Schulman et al. [1] construct a *surrogate objective* that can be optimised with the *old* samples. Define the importance-sampling ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (18)$$

The surrogate objective is

$$L^{\text{CPI}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right], \quad (19)$$

which matches $J(\theta)$ to first order around θ_{old} but can be optimised with fixed samples.

4.3 The KL Constraint

To prevent the new policy from straying too far from the old one, TRPO solves

$$\max_{\theta} L^{\text{CPI}}(\theta) \quad \text{subject to} \quad \mathbb{E}_t[\text{KL}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \parallel \pi_{\theta}(\cdot | s_t))] \leq \delta, \quad (20)$$

where δ is a small trust-region radius.

The constraint is enforced via a conjugate-gradient solve for the natural gradient direction followed by a line-search. This guarantees monotonic policy improvement but comes with significant implementation complexity:

- Requires second-order information (Fisher-vector products).
- Conjugate gradient and back-tracking line-search add engineering overhead.
- Difficult to apply to architectures with shared parameters between actor and critic.
- Not directly compatible with small-batch stochastic gradient descent.

These drawbacks motivate the simpler PPO family of algorithms.

5 From TRPO to PPO: Internalising the Constraint

5.1 Lagrangian Relaxation of TRPO

The constrained problem in Eq. (20) can be converted to an unconstrained one by introducing a Lagrange multiplier $\beta \geq 0$:

$$\max_{\theta} L^{\text{CPI}}(\theta) - \beta \mathbb{E}_t[\text{KL}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \parallel \pi_{\theta}(\cdot | s_t))]. \quad (21)$$

This *KL-penalty* formulation is an equivalent characterisation of TRPO: for the right value of β the unconstrained solution coincides with the constrained one.

The trouble is that β must be tuned—or adapted—to match the desired trust-region radius δ , and the appropriate value varies across environments and over the course of training. Adaptive schemes (increasing β when the KL exceeds δ and decreasing it otherwise) work in practice but introduce an additional hyper-parameter loop and make the algorithm more brittle.

More fundamentally, the KL penalty is a somewhat indirect handle on what we actually care about. This motivates asking: *what does the KL constraint protect against, and can we express that protection more directly in the objective?*

5.2 What Does the KL Constraint Actually Do?

The surrogate L^{CPI} relies on importance sampling: data collected under $\pi_{\theta_{\text{old}}}$ are reused to estimate the gradient for π_{θ} . This is valid as long as the two distributions are close, because the importance-sampling ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (22)$$

remains well-behaved. When $r_t \gg 1$ or $r_t \approx 0$, a single sampled transition dominates or vanishes in the estimate, and the surrogate diverges from the true objective.

The KL divergence is one measure of how far π_{θ} has moved from $\pi_{\theta_{\text{old}}}$, but it acts on the full distribution. At the level of individual action probabilities, what matters is simply that r_t stays close to 1. In fact, to first order

$$\text{KL}(\pi_{\theta_{\text{old}}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)) \approx \frac{1}{2} \mathbb{E}_{a \sim \pi_{\theta_{\text{old}}}} [(r_t(\theta) - 1)^2], \quad (23)$$

so the KL constraint is essentially a constraint on how much r_t can deviate from 1.

This suggests a simpler route: instead of penalising the KL divergence globally, *directly prevent r_t from leaving a symmetric interval $[1 - \varepsilon, 1 + \varepsilon]$ around 1.*

5.3 Deriving the Clipped Objective

Suppose we want to maximise $L^{\text{CPI}}(\theta) = \mathbb{E}_t[r_t(\theta) \hat{A}_t]$ subject to $r_t(\theta) \in [1 - \varepsilon, 1 + \varepsilon]$. The question is what happens to the gradient when r_t reaches the boundary of this interval.

We want a *pessimistic* objective: one that gives no gradient incentive to push r_t further outside the interval than it already is. Consider the two cases separately.

Case 1: $\hat{A}_t > 0$ (the action was better than average). We want to increase r_t —but only up to $1 + \varepsilon$. Beyond that point the update is too large to trust. An objective that captures this is

$$\min(r_t \hat{A}_t, (1 + \varepsilon) \hat{A}_t) = \min(r_t, 1 + \varepsilon) \hat{A}_t. \quad (24)$$

The gradient is positive (encouraging increasing r_t) when $r_t < 1 + \varepsilon$, and zero beyond it.

Case 2: $\hat{A}_t < 0$ (the action was worse than average). We want to decrease r_t —but only down to $1 - \varepsilon$. An objective that captures this is

$$\min(r_t \hat{A}_t, (1 - \varepsilon) \hat{A}_t) = \max(r_t, 1 - \varepsilon) \hat{A}_t. \quad (25)$$

Because $\hat{A}_t < 0$, the min over two negative quantities takes the one closest to zero, i.e. it stops the ratio from being pushed below $1 - \varepsilon$.

Unified form. Both cases are captured simultaneously by

$$\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t), \quad (26)$$

which equals $r_t \hat{A}_t$ inside the interval and flattens at the boundary regardless of the sign of $\hat{A}_t$. Taking the expectation over time steps yields the *clipped surrogate objective*

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right], \quad (27)$$

which is precisely the PPO objective introduced in Section 5.

Remark 5.1. L^{CLIP} is a lower bound on L^{CPI} in the region where $r_t \in [1 - \varepsilon, 1 + \varepsilon]$, and equals it there exactly. Outside the interval the clipping makes the bound loose in a direction that discourages further deviation—the objective becomes a conservative, pessimistic surrogate rather than an optimistic one. This is the key structural property that gives PPO its stability.

6 Proximal Policy Optimisation (PPO)

6.1 The Clipped Surrogate Objective

Schulman et al. [3] propose replacing the hard KL constraint with a soft, *clipped* surrogate objective that implicitly penalises large policy changes:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right], \quad (28)$$

where $\varepsilon > 0$ (typically 0.1–0.2) is the clipping coefficient.

The intuition is straightforward:

- When $\hat{A}_t > 0$, the gradient pushes r_t upward. The clip prevents the ratio from exceeding $1 + \varepsilon$, discouraging an excessively large update on a single positive sample.
- When $\hat{A}_t < 0$, the gradient pushes r_t downward. The clip prevents the ratio from falling below $1 - \varepsilon$.

Taking the min ensures we never benefit from moving r_t outside $[1 - \varepsilon, 1 + \varepsilon]$.

6.2 Value Function Loss

Alongside the policy, PPO trains a value function V_ϕ by minimising a clipped MSE loss:

$$L^{\text{VF}}(\phi) = \mathbb{E}_t \left[\max \left((V_\phi(s_t) - \hat{R}_t)^2, (\text{clip}(V_\phi(s_t), V_{\phi_{\text{old}}}(s_t) - \varepsilon_V, V_{\phi_{\text{old}}}(s_t) + \varepsilon_V) - \hat{R}_t)^2 \right) \right], \quad (29)$$

where $\hat{R}_t = \hat{A}_t + V_{\phi_{\text{old}}}(s_t)$ are the estimated returns and ε_V (often equal to ε) is the value clipping coefficient.

6.3 Entropy Bonus

An entropy bonus is added to encourage exploration:

$$L^{\text{ENT}}(\theta) = \mathbb{E}_t [H[\pi_\theta(\cdot | s_t)]] , \quad (30)$$

where $H[\cdot]$ denotes the Shannon entropy.

6.4 Full PPO Objective

The combined objective minimised at each update is

$$\boxed{L^{\text{PPO}}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_{\text{VF}} L^{\text{VF}}(\phi) - c_{\text{ENT}} L^{\text{ENT}}(\theta)}, \quad (31)$$

with scalar coefficients c_{VF} and c_{ENT} controlling the relative weighting of the three terms.

6.5 Implementation

PPO is attractive because it requires only first-order optimisation (e.g. Adam), makes multiple gradient steps over the same batch of collected data (controlled by `update_epochs`), and needs no conjugate gradient or second-order information. The clipping mechanism serves as a lightweight trust-region surrogate.

7 Extensions in PufferLib

PufferLib implements PPO with several extensions that improve sample efficiency and training stability. The main training loop lives in `pufferlib/pufferl.py` (`PufferRL.train()`, lines 332–476).

7.1 V-Trace Importance Weighting

When data are collected asynchronously, or when multiple gradient steps are taken over the same batch, the policy that collected the data ($\pi_{\theta_{\text{old}}}$) may differ from the current policy π_{θ} . V-Trace [4] corrects for this off-policy-ness by incorporating importance weights into the advantage computation.

Define the clipped importance ratios

$$\rho_t = \min\left(\bar{\rho}, \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}\right), \quad (32)$$

$$c_t = \min\left(\bar{c}, \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}\right). \quad (33)$$

The V-Trace target value is

$$v_t = V(s_t) + \sum_{k=t}^{t+n-1} \gamma^{k-t} \left(\prod_{i=t}^{k-1} c_i \right) \rho_k \delta_k, \quad (34)$$

where $\delta_k = r_{k+1} + \gamma V(s_{k+1}) - V(s_k)$ is the one-step TD error.

PufferLib unifies GAE (Eq. 17) and V-Trace (Eq. 27) into a single backward pass. Inspecting the CUDA kernel in `pufferlib/extensions/cuda/pufferlib.cu` (lines 7–20):

Listing 1: Unified GAE + V-Trace advantage kernel (CUDA, `pufferlib/extensions/cuda/pufferlib.cu`, lines 7–20).

```

1  __host__ __device__ void puff_advantage_row_cuda(
2      float* values, float* rewards, float* dones,
3      float* importance, float* advantages,
4      float gamma, float lambda,
5      float rho_clip, float c_clip, int horizon) {
6      float lastpufferlam = 0;
7      for (int t = horizon-2; t >= 0; t--) {
8          int t_next = t + 1;
9          float nextnonterminal = 1.0 - dones[t_next];
10         float rho_t = fminf(importance[t], rho_clip);    // Eq.
                (16)

```

```

11         float c_t    = fminf(importance[t], c_clip);          // Eq.
    (17)
12         float delta = rho_t * (rewards[t_next]
13                               + gamma * values[t_next] * nextnonterminal
14                               - values[t]);                    // rho
    -scaled TD error
15         lastpufferlam = delta
16                               + gamma * lambda * c_t
17                               * lastpufferlam * nextnonterminal;
18         advantages[t] = lastpufferlam;
19     }
20 }

```

The recursion is a backward sweep over the horizon. The `importance` buffer holds $\pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ ratios computed from the most recent forward pass. An identical CPU fallback lives in `pufferlib/extensions/pufferlib.cpp` (lines 28–41).

The function is invoked in the training loop (`pufferl.py`, lines 354–356):

Listing 2: Advantage computation call in `pufferlib/pufferl.py` (lines 352–356).

```

1 shape = self.values.shape
2 advantages = torch.zeros(shape, device=device)
3 advantages = compute_puff_advantage(
4     self.values, self.rewards,
5     self.terminals, self.ratio, advantages,
6     config['gamma'], config['gae_lambda'],
7     config['vtrace_rho_clip'], config['vtrace_c_clip'])

```

Relevant configuration values (from `pufferlib/config/default.ini`):

Parameter	Default	Description
<code>gamma</code>	0.995	Discount factor γ
<code>gae_lambda</code>	0.90	GAE λ
<code>vtrace_rho_clip</code>	1.0	$\bar{\rho}$ in Eq. (25)
<code>vtrace_c_clip</code>	1.0	$\bar{c}$ in Eq. (26)

Remark 7.1. With `vtrace_rho_clip = vtrace_c_clip = 1` the clipping has little effect (ratios are at most 1 before any update), making the estimator essentially equivalent to standard GAE immediately after data collection. The clipping becomes meaningful when multiple gradient epochs are performed over the same batch (`update_epochs > 1`), where the policy has already moved away from the behaviour policy.

7.2 Priority Experience Replay

Rather than sampling minibatches uniformly, PufferLib assigns a probability to each trajectory segment proportional to the magnitude of its advantage:

$$p_i \propto |A_i|^\alpha + \epsilon, \quad (35)$$

where α controls the degree of prioritisation and ϵ is a small floor.

To correct for the induced sampling bias, importance-sampling weights

$$w_i = (N \cdot p_i)^{-\beta} \quad (36)$$

are annealed from β_0 towards 1 over training. The implementation is in `pufferl.py` (lines 358–363):

Listing 3: Priority sampling in `pufferlib/pufferl.py` (lines 358–363).

```
1 # Prioritize experience by advantage magnitude
2 adv = advantages.abs().sum(axis=1)
3 prio_weights = torch.nan_to_num(adv**a, 0, 0, 0)
4 prio_probs = (prio_weights + 1e-6) / (prio_weights.sum() + 1e-6)
5 idx = torch.multinomial(prio_probs, self.minibatch_segments)
6 mb_prio = (self.segments * prio_probs[idx, None])**-anneal_beta
```

The minibatch advantages are then reweighted before the loss computation:

Listing 4: Priority-weighted advantage normalisation (`pufferl.py`, lines 407–409).

```
1 adv = mb_advantages
2 adv = mb_prio * (adv - adv.mean()) / (adv.std() + 1e-8)
```

Relevant configuration values:

Parameter	Default	Description
<code>prio_alpha</code>	0.8	Prioritisation exponent α
<code>prio_beta0</code>	0.2	Initial IS correction β_0

7.3 Clipped Surrogate and Value Losses

The PPO clipped surrogate objective (Eq. 21) is implemented directly in `pufferl.py` (lines 411–420):

Listing 5: PPO clipped policy and value losses (`pufferl.py`, lines 411–420).

```
1 # Policy gradient loss
2 pg_loss1 = -adv * ratio #
   unclipped
3 pg_loss2 = -adv * torch.clamp(ratio,
4                               1 - clip_coef, 1 + clip_coef) #
   clipped
5 pg_loss = torch.max(pg_loss1, pg_loss2).mean()
6
7 # Value function loss (with clipping)
8 newvalue = newvalue.view(mb_returns.shape)
9 v_clipped = mb_values + torch.clamp(
10     newvalue - mb_values, -vf_clip, vf_clip)
11 v_loss_unclipped = (newvalue - mb_returns) ** 2
12 v_loss_clipped = (v_clipped - mb_returns) ** 2
13 v_loss = 0.5 * torch.max(v_loss_unclipped, v_loss_clipped).mean()
```

The total loss combines all three terms (line 424):

Listing 6: Combined PPO loss (`pufferl.py`, line 424).

```
1 loss = pg_loss + config['vf_coef']*v_loss - config['ent_coef']*
   entropy_loss
```

Relevant configuration values:

Parameter	Default	Description
<code>clip_coef</code>	0.2	ε in Eq. (21)
<code>vf_clip_coef</code>	0.2	Value function clipping coefficient
<code>vf_coef</code>	2.0	c_{VF} in Eq. (24)
<code>ent_coef</code>	0.001	c_{ENT} in Eq. (24)

7.4 Gradient Clipping and Learning Rate Schedule

After computing the loss, gradients are accumulated over `accumulate_minibatches` steps and then clipped before the optimiser step (`pufferl.py`, lines 442–446):

Listing 7: Gradient clipping (`pufferl.py`, lines 442–446).

```

1 loss.backward()
2 if (mb + 1) % self.accumulate_minibatches == 0:
3     torch.nn.utils.clip_grad_norm_(
4         self.policy.parameters(), config['max_grad_norm'])
5     self.optimizer.step()
6     self.optimizer.zero_grad()

```

The learning rate is linearly annealed from `learning_rate` to `min_lr_ratio` $\times$ `learning_rate` over the course of training (controlled by a `CosineAnnealingLR` or linear scheduler, configured via `anneal_lr`).

Parameter	Default	Description
<code>max_grad_norm</code>	1.5	Gradient clip norm
<code>learning_rate</code>	0.015	Initial Adam learning rate
<code>anneal_lr</code>	True	Enable LR annealing
<code>min_lr_ratio</code>	0.0	Final LR as fraction of initial

8 Training Diagnostics: Understanding the Log

PufferLib prints and logs a dashboard of scalar metrics after each update. The relevant quantities are computed in `pufferl.py` (lines 396–457) and prefixed with `losses/` when sent to Wandb or Neptune.

Metric	Formula / source	Interpretation
<code>policy_loss</code>	$\mathbb{E}_t[\max(\dots)]$ (lines 412–414)	PPO clipped policy gradient loss. Magnitude alone is not diagnostic; watch for trends or spikes.
<code>value_loss</code>	$0.5 \mathbb{E}_t[\max(v_{\text{unc}}^2, v_{\text{clip}}^2)]$ (lines 418–420)	Critic MSE. Should decrease over training.
<code>entropy</code>	$\mathbb{E}_t[H[\pi_\theta(\cdot s_t)]]$ (line 422)	Policy entropy. Too low $\Rightarrow$ premature convergence / insufficient exploration.
<code>old_approx_kl</code>	$\mathbb{E}_t[-\log r_t(\theta)]$ (line 397)	First-order KL approximation using the old ratio. Should be small.
<code>approx_kl</code>	$\mathbb{E}_t[(r_t - 1) - \log r_t]$ (line 398)	Second-order KL approximation (Jensen–Shannon style). If consistently $\gg 0.01$, consider reducing learning rate or increasing clipping. Can be used for early stopping.
<code>clipfrac</code>	$\mathbb{E}_t[\mathbb{1}(r_t - 1 > \varepsilon)]$ (line 399)	Fraction of timesteps where clipping is active. Ideally ~ 0.1 – 0.3 ; higher values indicate the policy is being updated too aggressively.
<code>importance</code>	$\mathbb{E}_t[r_t(\theta)]$ (line 438)	Mean importance ratio. Should stay close to 1; large deviations indicate significant policy drift within a batch.
<code>explained_variance</code>	$1 - \text{Var}(\hat{R} - \hat{V}) / \text{Var}(\hat{R})$ (lines 453–456)	Fraction of return variance explained by the critic. Near 1 is ideal; near 0 or negative means the critic is no better than a constant baseline.

Remark 8.1. A healthy training run typically shows: `explained_variance` rising from near zero to > 0.9 ; `entropy` slowly decreasing; `approx_kl` staying below 0.02; and `clipfrac` in the range 0.1–0.3. Divergence in `value_loss` coupled with a drop in `explained_variance` is often an early warning of instability.

9 Summary

1. The utility $J(\theta)$ is the expected discounted return under π_θ .
2. The policy gradient theorem (REINFORCE) expresses $\nabla_\theta J$ as the expectation of the score function times the return.

3. Variance is reduced through reward-to-go, baseline subtraction, and GAE.
4. TRPO constrains the KL divergence to prevent catastrophic policy updates but requires second-order optimisation.
5. PPO replaces the hard KL constraint with a clipped surrogate objective, enabling first-order optimisation while retaining stability.
6. PufferLib extends PPO with a unified GAE + V-Trace advantage kernel, priority experience replay, clipped value loss, gradient accumulation, and learning rate annealing. All algorithmic components are concentrated in `pufferlib/pufferl.py` and `pufferlib/extensions/cuda/pufferlib.cu`.

References

- [1] Schulman, J., Levine, S., Moritz, P., Jordan, M., & Abbeel, P. (2015). *Trust Region Policy Optimization*. In *Proceedings of ICML 2015*, pp. 1889–1897.
- [2] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. In *Proceedings of ICLR 2016*.
- [3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
- [4] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., ... & Kavukcuoglu, K. (2018). *IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures*. In *Proceedings of ICML 2018*, pp. 1407–1416.
- [5] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). *Prioritized Experience Replay*. In *Proceedings of ICLR 2016*.